Center for Data Analytics and Modeling

Chuka University

Machine Learning Comprehensive Assignment

Multi-Model Classification for CVD Prediction using Python

Course: Machine Learning Methods
Instructor: Lumumba W. Victor
Email: lumumbavictor172@gmail.com
Date Issued: March 31, 2026
Due Date: 10th April 2026
Total Marks: 100
Language: Python 3.x

Learning Objectives

- Import and explore medical datasets using pandas
- Preprocess data for machine learning (cleaning, scaling, encoding)
- Implement train-test splitting and cross-validation with scikit-learn
- Fit multiple classification models: Logistic Regression, KNN, SVM, Random Forest
- Tune hyperparameters using GridSearchCV
- Evaluate and compare model performance using multiple metrics
- Interpret model coefficients, feature importance, and odds ratios
- Create professional visualizations for model comparison

Required Python Libraries

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, classification_report,
                             accuracy_score, roc_auc_score, roc_curve)
import warnings
warnings.filterwarnings('ignore')

```

1 General Instructions

- This assignment must be completed **individually**.
- Submit your work as a **PDF report** and the **Python script** (.py or .ipynb).
- All code must be well-commented and reproducible (set random_state = 2026).
- Use the provided dataset: `medical_data.csv`.
- Plots and tables must be clearly labeled with captions.
- Late submissions will incur a penalty of 10% per day.
- Plagiarism will result in zero marks for all parties involved.
- Use Python 3.8 or higher with scikit-learn 1.0 or higher.

Important Notes

- The target variable is **CVD** (Cardiovascular Disease), coded as binary (0 = No, 1 = Yes).
- Convert CVD to categorical for classification tasks.
- Some models (KNN, SVM) require feature scaling; others (RF, Logistic) benefit from it.
- Use the **same train-test split** for all models to ensure fair comparison.
- Set `random_state=2026` for reproducibility across all operations.

2 Dataset Description

The dataset `medical_data.csv` contains health indicators for 2,000 patients. The goal is to predict the presence of Cardiovascular Disease (CVD) using multiple machine learning models.

3 Tasks

3.1 Task 1: Data Import and Exploration (10 Marks)

1. Load all necessary libraries (pandas, numpy, matplotlib, seaborn, scikit-learn).
2. Import the dataset `medical_data.csv` using pandas.
3. Display the structure of the data (`df.info()`) and summary statistics (`df.describe()`).
4. Check for missing values and report the count and percentage per variable.
5. Visualize the distribution of the target variable **CVD** using a count plot.
6. Calculate and report the class imbalance ratio.

Variable	**Type**	**Description**
PatientID	Integer	Unique identifier
Age	Numeric	Age in years (18-80)
Gender	Categorical	Male / Female
BMI	Numeric	Body Mass Index (16-45)
SBP	Numeric	Systolic Blood Pressure (90-180 mmHg)
DBP	Numeric	Diastolic Blood Pressure (60-110 mmHg)
Cholesterol	Numeric	Total Cholesterol (120-320 mg/dL)
Glucose	Numeric	Fasting Glucose (70-200 mg/dL)
Smoking	Categorical	Never / Former / Current / Occasional
Alcohol	Categorical	None / Moderate / Heavy
Exercise	Categorical	Sedentary / Light / Moderate / Vigorous
FamilyHistory	Categorical	No / Yes
Medication	Categorical	No / Yes
SleepHours	Numeric	Hours per night (4-10)
StressLevel	Numeric	Scale 1-10
DietQuality	Numeric	Scale 1-5 (5 = best)
CVD	**Binary**	**Target (0 = No, 1 = Yes)**

Table 1: Variable Description for Medical Dataset

7. Create correlation heatmap for numeric variables.

Python Code Hint

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv("medical_data.csv")

# Check structure and missing values
df.info()
df.describe()
df.isnull().sum()

# Target distribution
sns.countplot(x='CVD', data=df)
plt.title('CVD Distribution')
plt.show()

# Correlation heatmap
numeric_cols = df.select_dtypes(include=[np.number]).columns
plt.figure(figsize=(12, 8))
sns.heatmap(df[numeric_cols].corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

3.2 Task 2: Data Preprocessing (15 Marks)

1. Handle missing values (justify your method: removal or imputation using SimpleImputer).
2. Convert categorical variables to numeric using one-hot encoding or label encoding:
   - Gender, Smoking, Alcohol, Exercise, FamilyHistory, Medication
3. Convert the target variable `CVD` to binary (0/1) if not already.
4. **For KNN and SVM:** Scale numeric predictors using StandardScaler (mean = 0, sd = 1).
5. **For Random Forest and Logistic Regression:** Scaling is optional but recommended.
6. Document your preprocessing decisions in the report.
7. **Important:** Fit scalers on training data only to avoid data leakage.

Python Code Hint

```python
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer

# Handle missing values
imputer = SimpleImputer(strategy='mean')
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)

# Encode categorical variables
categorical_cols = ['Gender', 'Smoking', 'Alcohol', 'Exercise',
                    'FamilyHistory', 'Medication']
for col in categorical_cols:
    le = LabelEncoder()
    df_imputed[col] = le.fit_transform(df_imputed[col])

# Separate features and target
X = df_imputed.drop(['PatientID', 'CVD'], axis=1)
y = df_imputed['CVD']

# Scale features (fit on training data only later)
scaler = StandardScaler()
```

3.3 Task 3: Data Partitioning (10 Marks)

1. Set the random state to `2026` for reproducibility.
2. Split the data into 70% training and 30% testing using `train_test_split`.
3. Use stratified sampling to preserve class balance.
4. Report the class distribution (counts and percentages) in both training and testing sets.
5. Verify that the class proportions are similar in both sets.
6. Save the indices for consistent splitting across all models.

Python Code Hint

```python
from sklearn.model_selection import train_test_split

# Set random state
RANDOM_STATE = 2026

# Stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=RANDOM_STATE,
    stratify=y  # Preserve class balance
)

# Verify class distribution
print("Training Set:")
print(y_train.value_counts())
print(y_train.value_counts(normalize=True))

print("\nTesting Set:")
print(y_test.value_counts())
print(y_test.value_counts(normalize=True))
```

3.4 Task 4: Cross-Validation Setup (10 Marks)

1. Define a cross-validation scheme with the following specifications:
   - Method: 10-fold cross-validation
   - Scoring metric: ROC AUC
   - Enable probability predictions for ROC analysis
2. Explain why cross-validation is important for model evaluation.
3. Create a function to evaluate models consistently across all algorithms.
4. Document the CV results for each model.

Python Code Hint

```python
from sklearn.model_selection import cross_val_score, StratifiedKFold

# Define 10-fold CV
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=RANDOM_STATE)

# Function to evaluate model with CV
def evaluate_with_cv(model, X, y, cv=cv):
    scores = cross_val_score(model, X, y, cv=cv, scoring='roc_auc')
    return {
        'mean_auc': scores.mean(),
        'std_auc': scores.std(),
        'scores': scores
    }

# Explain in report why CV is important
# (Reduces overfitting, provides stable performance estimates)
```

3.5 Task 5: Model 1 - Logistic Regression (15 Marks)

1. Train a Logistic Regression model using `sklearn.linear_model.LogisticRegression`.
2. Use L2 regularization with `C=1.0` as baseline.
3. Extract and report the model coefficients.
4. Calculate Odds Ratios (OR) for at least 3 significant predictors: $OR = e^{\beta}$
5. Interpret the Odds Ratios in the context of CVD risk.
6. Perform hyperparameter tuning using GridSearchCV for C values: [0.01, 0.1, 1, 10, 100].

Python Code Hint

```python
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
import numpy as np

# Scale features for logistic regression
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Logistic Regression
log_model = LogisticRegression(
    C=1.0,
    penalty='l2',
    solver='lbfgs',
    max_iter=1000,
    random_state=RANDOM_STATE
)
log_model.fit(X_train_scaled, y_train)

# Extract coefficients
coefficients = log_model.coef_[0]
odds_ratios = np.exp(coefficients)

# Hyperparameter tuning
param_grid = {'C': [0.01, 0.1, 1, 10, 100]}
grid_search = GridSearchCV(LogisticRegression(max_iter=1000),
                           param_grid,
                           cv=cv,
                           scoring='roc_auc')
grid_search.fit(X_train_scaled, y_train)
print("Best C:", grid_search.best_params_['C'])
```

3.6 Task 6: Model 2 - K-Nearest Neighbors (15 Marks)

1. Train a K-Nearest Neighbors model using `sklearn.neighbors.KNeighborsClassifier`.
2. Tune the parameter `n_neighbors` (k) using GridSearchCV: k = 1, 3, 5, ..., 21.
3. **Important:** Use the scaled dataset for KNN (distance-based algorithm).
4. Report the optimal k value selected by cross-validation.
5. Plot accuracy vs. k to visualize the tuning results.
6. Experiment with different distance metrics (euclidean, manhattan).

Python Code Hint

```python
from sklearn.neighbors import KNeighborsClassifier

# KNN requires scaled data
knn_model = KNeighborsClassifier()

# Hyperparameter tuning
param_grid = {
    'n_neighbors': list(range(1, 22, 2)),
    'metric': ['euclidean', 'manhattan']
}
grid_search = GridSearchCV(knn_model, param_grid, cv=cv,
    scoring='roc_auc')
grid_search.fit(X_train_scaled, y_train)

# View optimal k
print("Best k:", grid_search.best_params_['n_neighbors'])
print("Best metric:", grid_search.best_params_['metric'])

# Plot accuracy vs k
k_values = list(range(1, 22, 2))
cv_scores = []
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train_scaled, y_train, cv=
    cv, scoring='roc_auc')
    cv_scores.append(scores.mean())

plt.plot(k_values, cv_scores, marker='o')
plt.xlabel('k')
plt.ylabel('ROC AUC')
plt.title('KNN: ROC AUC vs k')
plt.show()
```

3.7 Task 7: Model 3 - Support Vector Machine (15 Marks)

1. Train a Support Vector Machine model using `sklearn.svm.SVC`.
2. Use RBF kernel with probability=True for ROC analysis.
3. Tune the hyperparameters using GridSearchCV:
   - `C` (cost): 0.1, 1, 10, 100
   - `gamma`: 0.001, 0.01, 0.1, 1
4. **Important:** Use the scaled dataset for SVM.
5. Report the optimal C and gamma values.
6. Explain the role of the kernel function in SVM.

Python Code Hint

```python
from sklearn.svm import SVC

# SVM requires scaled data
svm_model = SVC(kernel='rbf', probability=True, random_state=
    RANDOM_STATE)

# Hyperparameter tuning
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [0.001, 0.01, 0.1, 1]
}
grid_search = GridSearchCV(svm_model, param_grid, cv=cv,
    scoring='roc_auc')
grid_search.fit(X_train_scaled, y_train)

# View optimal parameters
print("Best C:", grid_search.best_params_['C'])
print("Best gamma:", grid_search.best_params_['gamma'])

# Plot tuning results (optional)
# Use pandas to create heatmap of CV results
```

3.8 Task 8: Model 4 - Random Forest (15 Marks)

1. Train a Random Forest model using `sklearn.ensemble.RandomForestClassifier`.
2. Tune the parameter `max_features`: 'sqrt', 'log2', None.
3. Set `n_estimators = 100` (number of trees) as baseline.
4. Report the optimal parameters.
5. Extract and plot feature importance.
6. Identify the top 5 most important predictors for CVD.
7. Experiment with n_estimators: [50, 100, 200].

Python Code Hint

```python
from sklearn.ensemble import RandomForestClassifier

# Random Forest (scaling not required but doesn't hurt)
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1  # Use all CPU cores
)

# Hyperparameter tuning
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_features': ['sqrt', 'log2', None]
}
grid_search = GridSearchCV(rf_model, param_grid, cv=cv,
    scoring='roc_auc')
grid_search.fit(X_train, y_train)  # No scaling needed

# View optimal parameters
print("Best n_estimators:", grid_search.best_params_['
    n_estimators'])
print("Best max_features:", grid_search.best_params_['
    max_features'])

# Feature importance
importances = grid_search.best_estimator_.feature_importances_
feature_names = X.columns
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
}).sort_values('Importance', ascending=False)

print(importance_df.head(10))
```

3.9 Task 9: Model Evaluation and Comparison (20 Marks)

1. Predict the class and probabilities on the **test set** for all 4 models.
2. For each model, generate a Confusion Matrix and report:
   - Accuracy
   - Sensitivity (Recall)
   - Specificity
   - Precision (PPV)
   - F1-Score
   - ROC AUC

3. Plot ROC curves for all 4 models on the same graph.
4. Calculate and compare AUC values for all models.
5. Create a comparison table summarizing all metrics.
6. Rank the models from best to worst based on AUC.
7. Perform statistical comparison using paired t-test on CV scores.

Python Code Hint

```python
from sklearn.metrics import (confusion_matrix, classification_report,
                             accuracy_score, roc_auc_score, roc_curve)

# Function to evaluate all metrics
def evaluate_model(model, X_test, y_test, scaled=False):
    if scaled:
        y_pred = model.predict(X_test)
        y_pred_proba = model.predict_proba(X_test)[:, 1]
    else:
        y_pred = model.predict(X_test)
        y_pred_proba = model.predict_proba(X_test)[:, 1]

    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()

    return {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Sensitivity': tp / (tp + fn),
        'Specificity': tn / (tn + fp),
        'Precision': tp / (tp + fp),
        'F1-Score': 2 * tp / (2 * tp + fp + fn),
        'ROC AUC': roc_auc_score(y_test, y_pred_proba)
    }

# Evaluate all models
results = {
    'Logistic Regression': evaluate_model(log_model, X_test_scaled, y_test, scaled=True),
    'KNN': evaluate_model(knn_best, X_test_scaled, y_test, scaled=True),
    'SVM': evaluate_model(svm_best, X_test_scaled, y_test, scaled=True),
    'Random Forest': evaluate_model(rf_best, X_test, y_test, scaled=False)
}

# Create comparison table
results_df = pd.DataFrame(results).T
print(results_df)
```

3.10 Task 10: ROC Curve Comparison (10 Marks)

1. Plot ROC curves for all 4 models on the same graph with different colors.
2. Include the diagonal reference line (random classifier).
3. Add legend with AUC values for each model.

4. Save the plot as high-resolution image (300 DPI).
5. Interpret the ROC curves and AUC values.
6. Discuss which model has the best discrimination ability.

Python Code Hint

```python
# ROC Curve Comparison
plt.figure(figsize=(10, 8))

models = [
    ('Logistic Regression', log_model, X_test_scaled, True),
    ('KNN', knn_best, X_test_scaled, True),
    ('SVM', svm_best, X_test_scaled, True),
    ('Random Forest', rf_best, X_test, False)
]

for name, model, X_t, scaled in models:
    if scaled:
        y_pred_proba = model.predict_proba(X_t)[:, 1]
    else:
        y_pred_proba = model.predict_proba(X_t)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
    auc = roc_auc_score(y_test, y_pred_proba)
    plt.plot(fpr, tpr, label=f'{name} (AUC = {auc:.3f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve Comparison: All Models')
plt.legend(loc='lower right')
plt.grid(alpha=0.3)
plt.savefig('roc_comparison.png', dpi=300, bbox_inches='tight')
plt.show()
```

3.11 Task 11: Interpretation and Recommendations (15 Marks)

1. Based on your results, answer the following:
   - Which model performed best? Justify using metrics.
   - Which model performed worst? Explain possible reasons.
   - Was feature scaling necessary for all models? Explain.
   - Did cross-validation improve model reliability? Explain.
   - How do your results compare with literature on CVD prediction?
2. For the best model, identify the top 3 risk factors for CVD.
3. Calculate and interpret odds ratios for logistic regression coefficients.

4. Discuss the trade-offs between model accuracy and interpretability.
5. Provide recommendations for healthcare practitioners based on your findings.
6. Discuss any limitations of your analysis (e.g., observational data, potential confounders).
7. Suggest future improvements (e.g., ensemble methods, deep learning, external validation).

Report Guidelines

Your report should include:

- Executive summary (1 paragraph)
- Methodology description (including Python version and library versions)
- Results with tables and figures (all plots must have titles and labels)
- Model comparison table with all metrics
- ROC curve comparison figure
- Feature importance analysis
- Interpretation and recommendations
- Limitations and future work
- Appendix with complete Python code
- References (if you used external resources)

4 Submission Guidelines

- **File Format:** Submit a single ZIP file named `ML_Assignment_YourName.zip`.
- **Contents:**
  1. Report (PDF) containing answers, plots, tables, and interpretations (max 20 pages).
  2. Python Script (.py) or Jupyter Notebook (.ipynb) with all code and comments.
  3. Requirements file (requirements.txt) listing all package versions.
  4. The dataset (if modified).
  5. All generated figures (PNG format, 300 DPI).
- **Naming Convention:** `YourName_StudentID_ML_Assignment.pdf`
- **Submission Portal:** Upload to the CDAM Learning Management System.
- **Deadline:** [Insert Date and Time]
- **Late Policy:** 10% penalty per day, maximum 3 days late.

5 Grading Rubric

Component	**Marks**	**Criteria**
Data Import & Exploration	10	Correct loading, summary, visualization, missing value report
Preprocessing	15	Proper handling of NA, encoding, scaling for appropriate models
Data Partitioning	10	Correct 70/30 split, stratification, random state set, distribution report
Cross-Validation Setup	10	Correct CV setup, explanation of importance, consistent evaluation
Logistic Regression	15	Model fitting, coefficients, odds ratios, interpretation, tuning
KNN Model	15	Model fitting, k tuning, scaled data usage, optimal k report, visualization
SVM Model	15	Model fitting, C/gamma tuning, scaled data usage, kernel explanation
Random Forest	15	Model fitting, parameter tuning, variable importance, top predictors
Model Evaluation	20	Confusion matrices, ROC curves, AUC comparison, metrics table, statistical comparison
Interpretation	15	Model ranking, risk factors, odds ratios, recommendations, limitations
Total	**100**	

Table 2: Assignment Grading Rubric

6 Tips for Success

Best Practices

- **Start Early:** This assignment requires training 4 models; allow time for computation.
- **Use Comments:** Comment your code to explain each step (helps with partial marks).
- **Check Scaling:** Remember KNN and SVM need scaled data; RF and Logistic benefit from it.
- **Same Split:** Use the same train-test split for all models to ensure fair comparison.
- **Save Objects:** Save your trained models using pickle/joblib to avoid re-running.
- **Visualize:** Include ROC curves, feature importance plots, and tuning plots.
- **Interpret:** Don't just report numbers; explain what they mean for CVD prediction.
- **Proofread:** Check your report for clarity, grammar, and proper labeling of figures.
- **Version Control:** Use Git to track changes in your code.
- **Document Dependencies:** Include requirements.txt with package versions.

Common Mistakes to Avoid

- **Data Leakage:** Do not scale using test data; fit scaler on training data only.
- **Wrong Target Type:** CVD must be numeric (0/1) for scikit-learn classification.
- **Inconsistent Splits:** Using different splits for different models invalidates comparison.
- **Missing Metrics:** Report multiple metrics (not just accuracy) for imbalanced data.
- **No Interpretation:** Numbers without context earn fewer marks.
- **Uncited Code:** If using online resources, cite them appropriately.
- **Ignoring Warnings:** Address convergence warnings in logistic regression.
- **Overfitting:** Monitor training vs validation performance.

7 Additional Resources

- **scikit-learn Documentation:** https://scikit-learn.org/stable/
- **Python Data Science Handbook:** https://jakevdp.github.io/PythonDataScienceHandbo
- **Introduction to Statistical Learning (Python):** https://www.statlearning.com/
- **CDAM Resources:** https://cdam.chuka.ac.ke
- **Kaggle Learn:** https://www.kaggle.com/learn
- **Office Hours:** Tuesdays 2-4 PM, Science Block Room 205
- **Python Setup Guide:** https://cdam.chuka.ac.ke/python-setup

8 Python Environment Setup

Required Packages

```
# requirements.txt
pandas>=1.5.0
numpy>=1.23.0
scikit-learn>=1.2.0
matplotlib>=3.6.0
seaborn>=0.12.0
jupyter>=1.0.0
```

Installation:

```
pip install -r requirements.txt
```

Verify Installation:

```
import sklearn
print(sklearn.__version__)
```

9 Academic Integrity Statement

Important

By submitting this assignment, you declare that:

1. This work is your own individual effort.
2. You have not copied code or content from other students.
3. All external sources (websites, books, tutorials) are properly cited.
4. You understand that plagiarism will result in disciplinary action.
5. You have followed all ethical guidelines for data analysis.

Citation Example:

- If you used scikit-learn documentation: "scikit-learn developers (2023). scikit-learn: Machine Learning in Python. `https://scikit-learn.org`"
- If you adapted code from Stack Overflow: Include URL and access date.

Center for Data Analytics and Modeling (CDAM)
Chuka University
"Data Driven Decisions for Sustainable Development"
https://cdam.chuka.ac.ke

This assignment is designed to assess practical machine learning skills.
All work must be your own. Academic integrity is strictly enforced.
Python 3.x | scikit-learn 1.x | pandas 1.x